

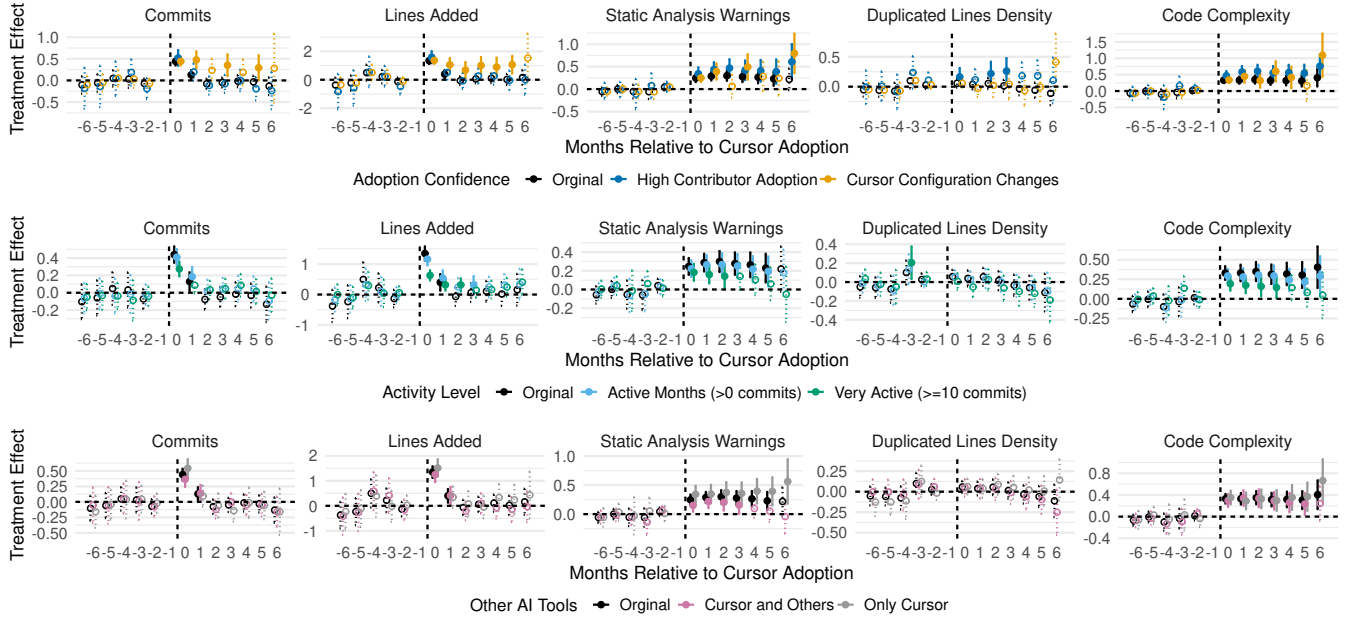


Figure 4: The estimated “horizon-average” treatment effects in alternative dataset settings as robustness checks. Row 1: Robustness check on subsets with higher Cursor adoption confidence, showing generally stronger effects in high-confidence adoption observations. Row 2: Robustness check on subsets of different activity levels, showing the persistence of our findings even in very active repositories. Row 3: Robustness check between repositories with/without observable use of other AI tools, showing that the potential use of other AI tools weakens Cursor effect estimations, but only by a small margin.

4.3 Robustness Checks

The above overall findings and the internal validity concerns behind them (Section 3.5) present significant interpretation challenges. In this section, we present additional robustness checks to explore and rule out alternative explanations for our observed results.

Recall that one limitation of our identification strategy is that it merely displays an “intent-to-treat” (ITT) signal without us knowing how Cursor was actually used in each treatment repository (Section 3.5). To test whether our findings are driven by repositories with genuine, sustained usage (versus minimal engagement), we repeat the same DiD modeling process on two subsets of data based on two alternative measurements of usage intensity:

- **High Contributor Adoption:** For each repository, we identify contributors modifying Cursor files (indicating experimentation) and calculate their fraction of total repository commits during observation. This subset keeps only “high-adoption” repositories, in which those modifying Cursor files also contributed $\geq 80\%$ of commits, along with their own matched controls.
- **Cursor Configuration Changes:** For each repository, we identify all commits where `.cursorrules` files were modified post-adoption. This subset keeps only post-adoption observations with at least one commit modifying Cursor files during that period, representing sustained usage and ruling out potential post-adoption abandonments.

Results from the two subsets (Figure 4, Row 1) show that our quality-related findings are robust and amplified in subsamples with higher sustained usage: The accumulation of static analysis

warnings and code complexity is stronger—not weaker—among repositories with continued configuration refinement and where Cursor users dominated activity. This strengthens causal interpretation: Effects are attributable to Cursor adoption rather than spurious correlation with other changes, and our main ITT estimates likely understate effects among repositories with intensive, sustained usage. Interestingly, while the velocity gain remains transient on average across repositories with high contributor adoption, we still observe a velocity increase in observations where developers are actively modifying Cursor files. This indicates that abandonments of Cursor post-adoption likely nullify at least part of the velocity gains (more discussions in Section 5).

Another alternative explanation for the transient velocity gain is that our DiD setting still does not sufficiently control for the macro-trend that many open-source repositories become inactive very fast (e.g., within two months). To test against this alternative explanation, we build two subsets of observations: The first keeps only repository-months with at least one commit, and the second keeps only repository-months with at least 10 commits. The results (Figure 4, Row 2) show that our main findings persist even in very active repositories, albeit weaker than our main ITT estimates, which is expected because it would be more difficult to achieve the same percentage change in the measured outcomes in repositories with higher baseline sizes and activity levels.

Finally, we assess the impact of dataset contamination from alternative AI coding tools in our study setting (Section 3.5). For this purpose, we take an overly conservative approach and identify all